

NetSpectre Pro (C++ Edition)

NetSpectre Pro is a terminal-style CLI application written in C++. It provides: - a command shell with user/developer modes - an AkteViewer workflow for encrypted `.persondata` files - an optional local JARVIS chat mode (`jarvis-local`)

Security note: AkteViewer currently uses a custom letter-shift cipher. This is obfuscation, not cryptographic security.

Access / Login

Credentials are not public. If you need access, request a username/password from:

Hendrik Hanking — `hendrik.hanking@icloud.com`

Account Tiers

- **User**: default account commands
- **Pro Basic**: includes User commands + Pro Basic commands
- **Developer**: includes User + Pro Basic + Developer commands

Quick Start

macOS

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
./build/release/Netspro
```

Windows 11 (PowerShell)

```
cmake -S . -B build -G "Visual Studio 17 2022" -A x64
cmake --build build --config Release
.\build\release\Netspro.exe
```

Detailed Installation

1. Install prerequisites

macOS

1. Install Xcode Command Line Tools: `bash xcode-select --install`
2. Install Homebrew (if not installed):
`bash /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"`
3. Install required tools: `bash brew install cmake python`
4. Verify tools: `bash cmake --version python3 --version`

Windows 11 (PowerShell)

1. Install Visual Studio C++ Build Tools:

```
powershell winget install --id Microsoft.VisualStudio.2022.BuildTools -e
```

2. Install CMake: `powershell winget install --id Kitware.CMake -e`

3. Install Python 3: `powershell winget install --id Python.Python.3.13 -e`

4. Restart PowerShell, then verify tools: `powershell cmake --version py --version`

2. Build with CMake

macOS

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release  
cmake --build build -j
```

Windows 11 (PowerShell)

```
cmake -S . -B build -G "Visual Studio 17 2022" -A x64  
cmake --build build --config Release
```

3. Run the program

macOS

```
./build/release/Netspro
```

Windows 11 (PowerShell)

```
.\build\release\Netspro.exe
```

Optional: Enable Local JARVIS (`jarvis-local`)

Use this only if you want local AI chat in-app. Core commands (`info` , `record` , `help` , `exit`) do not require this.

macOS

```
python3 -m pip install --upgrade pip ollama  
brew install --cask ollama  
ollama pull qwen2:1.5b  
ollama pull phi3.5:latest  
ollama pull mistral:latest
```

Windows 11 (PowerShell)

```
py -m pip install --upgrade pip ollama  
irm https://ollama.com/install.ps1 | iex  
ollama pull qwen2:1.5b  
ollama pull phi3.5:latest  
ollama pull mistral:latest
```

If the install script fails, run PowerShell as Administrator and retry.

Alternative fallback: install Ollama directly from <https://ollama.com/download/windows> or try:

```
winget install --id Ollama.Ollama -e --accept-package-agreements --accept-source-agreements
```

Then start NetSpectre Pro and run:

```
jarvis-local
```

Commands

User commands

- `info`
Shows app/version/serial/dev mode information.
- `cls` / `clear`
Clears the terminal output.
- `cls -l` / `clear -l`
Clears the terminal and redraws the logo/main screen.
- `record`
Opens AkteViewer interactive mode (ask/open/create flow).
- `record new`
Creates a new encrypted `.persondata` file and opens it.
- `record open <Name>`
Opens `<Name>.persondata` directly.
- `jarvis-local`
Starts local JARVIS chat mode.
- `help`
Prints command help.
- `exit`
Exits the application.

Developer commands

- `devinfo`
Shows developer status and logged-in user.
- `devmode on|off`
Toggles dev mode (developer login required).
- `devtools`
Lists available developer tool commands.

Pro Basic commands

- `proinfo`
Shows Pro Basic account status and current user.
- `prostatus`
Shows currently available Pro Basic feature set.

Build Notes

- The project builds via CMake and outputs the executable to `build/release/`.

- `CMakeLists.txt` currently sets `CMAKE_CXX_STANDARD` to `14` .
- Post-build, `Plugins/JARVIS/AI.py` is copied into the release plugin folder.

Platform Notes

- **Windows:** reads BIOS serial using PowerShell `Get-CimInstance Win32_BIOS` .
- **macOS:** reads serial via `system_profiler` .
- **Linux:** reads `/sys/class/dmi/id/product_serial` .

Data Format / AkteViewer

- Stored data uses a shift pattern: `{9,9,4,13,2,10,3}` .
- `record new` creates file metadata and key seed content.
- `record open` decrypts, displays content, and allows appending lines.
- Empty line in editor mode saves and exits.

Troubleshooting

- If network-dependent tools fail (e.g., model download), retry after verifying connectivity.
- On Linux, serial read may require permission for `/sys/class/dmi/id/product_serial` .
- If `jarvis-local` fails, verify:
 - Ollama is installed and running
 - required models are pulled
 - Python package `ollama` is installed
 - on Windows, Ollama install completed successfully (`irm https://ollama.com/install.ps1 | iex`)

License / Copyright

COPYRIGHT Hendrik Vincent Hanking 2025. All rights reserved.